Summary in Graph

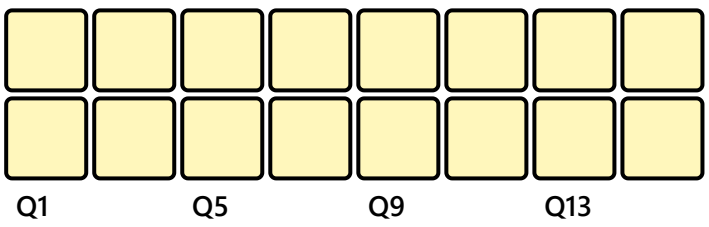
Exam Summary (GO Classes GATE CS | Operating System | Process Synchronization | Weekly Quiz 1)

Qs. Attempted:	0 (0 + 0)
Correct Attempts:	0 (0 + 0)
Incorrect Attempts:	0 (0 + 0)
Correct Marks:	0 (0 + 0)
Penalty Marks:	0 (0 + 0)
Resultant Marks:	0 (0 + 0)

Total Questions:	16 (7 + 9)
Total Marks:	25 (7 + 18)
Exam Duration:	48 Minutes
Time Taken:	0 Minutes

Individual Question Statistics

Select Metric: Marks Obtained



* Average Stats based on first-attempt stats of Top 10% (35) out of 347 users

EXAM RESPONSE | EXAM STATS | FEEDBACK

Technical

Q #1 | Multiple Select Type | Award: 1 | Penalty: 0 | Operating System

Consider 3 processes having synchronization primitives with counting semaphores L and R.

```
semaphore L = 3, R = 0;   /* initialization */

/* Process 1 */
while(1){
 P(L);
print('C');
 V(R);
 }

/* process 2 */
while(1){
 P(R);
print('A');
print('B');
 V(R);
 }

/* process 3 */
while(1){
 P(R);
print('D');
 }
```

Which of the following is False?

A. CABACD is possible output
B. CABABD is possible output
C. CDABCA is possible output
D. CCADBC is possible output

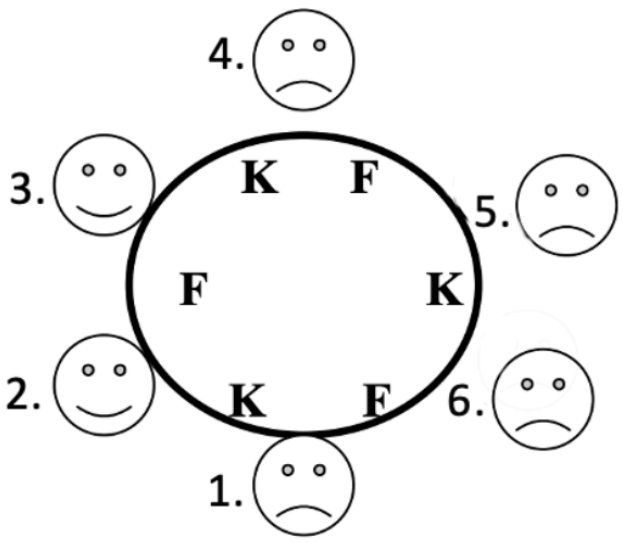
Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #2 | Multiple Select Type | Award: 1 | Penalty: 0 | Operating System

The notorious dining gourmand problems is as follows:

Six gourmands are seated around a table, with a substantial portion of perfectly cooked roast vegetables at the center.
Forks (represented as 'F') and knives (represented as 'K') are arranged as shown below. (Gourmands who prefer their roasted vegetables less cooked wear displeased expressions.) Each gourmand obeys the following algorithm:

1. Grab the closest knife.
2. Grab the closest fork.
3. Eat roasted vegetables
4. Put down the knife and fork where they used to be



Which statement(s) among the following is/are TRUE regarding the dining gourmand problem presented above?

A. The possibility of deadlock exists in the given scenario.
B. The possibility of deadlock does NOT exist in the given scenario.
C. Deadlock can occur if we swap steps 1 and 2 in the algorithm for odd philosophers while keeping the algorithm unchanged for even philosophers.
D. Deadlock can occur if we swap steps 1 and 2 in the algorithm for even philosophers while keeping the algorithm unchanged for odd philosophers.

Your Answer: | Correct Answer: B;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #3 | Multiple Choice Type | Award: 1 | Penalty: 0.33 | Operating System

A parking can accommodate up to 20 cars. There is one gate through which all cars must go one at a time when they enter or leave the parking.
Add the required semaphore calls to the following template to ensure the conditions:

a. The parking will never contain more than 20 cars, and
b. Cars entering or leaving the parking will go through the gate one at a time.

```
semaphore spaces = 20;
semaphore gate = 1;
enter_parking()
{
    X:
    Y:
    go_through_gate();
    V(gate);
}// enter_parking

leave_parking()
{
    P(gate);
    go_through_gate();
    V(gate);
    Z:
}// leave_parking
```

Fill the terms in place of X, Y, and Z such that given conditions a, and b are satisfied.
"Blank" in the options represents no term needed.

A. X: $\mathrm{P(spaces)}$, Y: $\mathrm{P(gate)}$, Z: $\mathrm{V(spaces)}$
B. X: $\mathrm{P(gate)}$, Y: Blank, Z: $\mathrm{V(spaces)}$
C. X: $\mathrm{P(spaces)}$, Y: $\mathrm{V(spaces)}$, Z: $\mathrm{V(gate)}$
D. X: Blank, Y: $\mathrm{P(spaces)}$, Z: $\mathrm{V(gate)}$

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #4 | Multiple Select Type | Award: 1 | Penalty: 0 | Operating System

Which of the following are true about semaphores?

A. Semaphore $\mathrm{V}()$ increments the value of the semaphore and wakes a sleeping thread if the value of the semaphore is $<= 0$
B. Semaphores can NOT be used for locking.
C. The interface for Semaphore $\mathrm{P}()$ is specified in a way that prevents its implementation from busy waiting, even for a brief period of time.
D. The pure semaphore interface does not allow querying for the current value of the semaphore.

Your Answer: | Correct Answer: A;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #5 | Multiple Select Type | Award: 1 | Penalty: 0 | Operating System

Suppose a program has three threads Thread1, Thread2 and Thread3 and a shared counter, count, as shown below:

```
int count = 10;

Thread1(...)
{ // do something
    Wait(Lock);
    count++;
    Signal(Lock);
}

Thread2(...)
{ // do something
    Wait(Lock);
    count --;
    Signal(Lock);
}

Thread3(...)
{ // do something
    Wait(Lock);
    print(count);
    Signal(Lock);
}
```

What is the possible output that get printed after the completion of all threads? (Mark all the outputs which are possible)

A. 8
B. 9
C. 10
D. 11

Your Answer: | Correct Answer: B;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #6 | Numerical Type | Award: 1 | Penalty: 0 | Operating System

In the given implementation of a critical section, where 'mutex' is a binary semaphore initialized as 1 and 'x' is a shared integer initialized as 0.

```
down (mutex);
if (x == 0) {
    up (mutex);
    down (mutex);
    x++;
}
up (mutex);
```

What is the maximum possible final value of 'x' when 10 threads are concurrently executing the same code?

Your Answer: | Correct Answer: 10 | Not Attempted | Time taken: 00min 00sec | Discuss

Q #7 | Multiple Select Type | Award: 1 | Penalty: 0 | Operating System

Which of the following is/are TRUE?

A. Once a thread has acquired a lock $(\text{using } \mathrm{P}())$, it cannot be preempted by the scheduler until it has released the lock by calling $\mathrm{V}()$.
B. Peterson's algorithm can be used to enforce mutual exclusion on a multiprocessor.
C. Peterson's algorithm assumes the existence of a test-and-set instruction.

D. In a uniprocessor environment, when multiple threads exist within a single process, synchronization among the threads is unnecessary, even if they access shared data.

Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 00sec | Discuss

Q #8 | Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

Consider a concurrent program with many threads. There are two types of threads. Type A threads repeatedly call ProcedureA, shown below. Type B threads repeatedly call ProcedureB. These two procedures are synchronized using three semaphores, semX, semY, and semZ. You should assume that semX and semY each have an initial value of 1, and that semZ has an initial value of 2. ProcedureA and ProcedureB also make calls to ProcedureX, ProcedureY, and ProcedureZ. These procedures are not shown. However, they do not make any use of synchronization primitives.

`ProcedureA() {`
`  P(semX);`
`  ProcedureX();`
`  P(semZ);`
`  V(semX);`
`  ProcedureZ();`
`  V(semZ);`
`}`
	`ProcedureB() {`
	`  P(semY);`
	`  ProcedureY();`
	`  P(semZ);`
	`  V(semY);`
	`  ProcedureZ();`
	`  V(semZ);`
	`}`

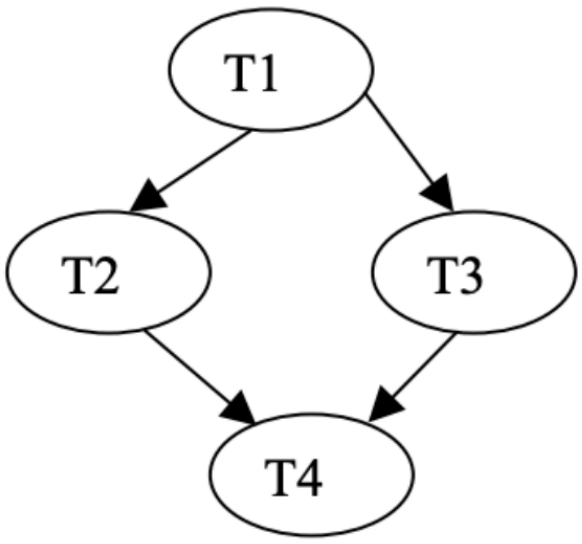
Which of the following is/are TRUE?

A. It is possible that two threads can execute ProcedureX() concurrently
B. It is possible that some thread will execute ProcedureX() at the same time that another thread is executing ProcedureY()
C. It is possible that some Type A thread will execute ProcedureX() at the same time that another Type A thread is executing ProcedureZ()
D. It possible that a deadlock will occur among these threads?

Your Answer: | Correct Answer: B;C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #9 | Numerical Type | Award: 2 | Penalty: 0 | Operating System

Assume you are given a graph that represents the relationship between four threads $(\mathrm{T1}, \mathrm{T2}, \mathrm{T3}, \mathrm{T4})$. An arrow from one thread (Tx) to another (Ty) means that thread Tx must finish its computation before Ty starts. Assume that the threads can arrive in any order.



How Many MINIMUM number of counting semaphores are needed to ensure the above dependency order of execution always holds?

Your Answer: | Correct Answer: 2 | Not Attempted | Time taken: 00min 02sec | Discuss

Q #10 Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

Consider three concurrently executing threads in the same process using two semaphores $s1$ and $s2$. Assume $s1$ has been initialized to 1, while $s2$ has been initialized to 0.

```
/* thread A */      /* thread B */      /* thread C */
  P(s2);            P(s1);              P(s1);
  P(s1);            x = x*x;            x = x+3;
  x = x*2;          V(s1);              V(s2);
  V(s1);                                V(s1);
```

What are the possible values of the global variable x, initialized to 0, after all three threads have terminated?

A. $x = 6$
B. $x = 12$
C. $x = 18$
D. $x = 36$

Your Answer: | Correct Answer: A;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #11 Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

```
   shared Integer total = 0;
   Process A
   for i = 1 to 3 do
      total = total + 1
5. Process B
   for j = 1 to 3 do
      total = total + 1
```

Assume that total must be loaded into a register before being incremented, and then is written back afterwards.

What are all the possible values that total can have when both threads complete?

A. 1
B. 3
C. 5
D. 6

Your Answer: | Correct Answer: B;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #12 | Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

Here is the classic semaphore version of the producer/consumer problem:

```
void  producer() {  // core of producer
  for (i = 0; i < num; i++) {
    sem_wait(empty);
    sem_wait(mutex);

    // produce

    sem_post(mutex);
    sem_post(full);
  }
}

void  consumer() {  // core of consumer
  while (!done) {
    sem_wait(full);
    sem_wait(mutex);

    // consume

    sem_post(mutex);
    sem_post(empty);
    // do something with tmp ...
  }
}
```

For the following statements about this working solution, which statement(s) is/are TRUE?

A. The semaphore empty must be initialized to 1
B. The semaphore empty can be initialized to 1
C. The semaphore full must be initialized to 1
D. The semaphore mutex must be initialized to 1

Your Answer: | Correct Answer: B;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #13 | Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

One way to avoid deadlock is to schedule threads carefully.
Assume the following characteristics of threads $\mathrm{T1}$, $\mathrm{T2}$, and $\mathrm{T3}$:

- $\mathrm{T1}$ (at some point) acquires and releases locks $\mathrm{L1}$, $\mathrm{L2}$
- $\mathrm{T2}$ (at some point) acquires and releases locks $\mathrm{L1}$, $\mathrm{L3}$
- $\mathrm{T3}$ (at some point) acquires and releases locks $\mathrm{L3}$, $\mathrm{L1}$, and $\mathrm{L4}$

Which of the schedules listed below ensures that deadlock can NOT occur?

A. $\mathrm{T1}$ runs to completion, then $\mathrm{T2}$ to completion, then $\mathrm{T3}$ runs
B. $\mathrm{T1}$ and $\mathrm{T2}$ run concurrently to completion, then $\mathrm{T3}$ runs
C. $\mathrm{T3}$ runs to completion, then $\mathrm{T1}$ and $\mathrm{T2}$ run concurrently
D. $\mathrm{T1}$ and $\mathrm{T3}$ run concurrently to completion, then $\mathrm{T2}$ runs

Your Answer: | Correct Answer: A;B;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #14 Multiple Select Type | Award: 2 | Penalty: 0 | Operating System

Given a basic spin lock, assume that locking the spin lock takes A time units (if no one is holding the lock); unlock also takes A time units. Furthermore, let C represent the time it takes for a context switch, and T be the maximum time any thread can spend in busy wait before yielding the CPU.
That means if a thread is doing a busy wait then it can wait for a maximum of T time after that it gives up the CPU voluntarily.

Assume this code sequence, executed by two threads on one processor at roughly the same time:

```
mutex_lock();
do_something(); // takes no time to execute
mutex_unlock();
exit();         // takes no time to execute
```

What is the worst-case time for the two threads to finish this code sequence? Assume that only three context switches can occur at a maximum.

A. $4\ \mathrm{A}+\mathrm{C}$
B. $4\ \mathrm{A}+3\ \mathrm{C}$
C. $4\ \mathrm{A}+\mathrm{C}+3\ \mathrm{T}$
D. $4\ \mathrm{A}+3\ \mathrm{C}+\mathrm{T}$

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #15 Numerical Type | Award: 2 | Penalty: 0 | Operating System

Assume you are building a special operating system that requires executing the expression:

```
z = F3(F1(x), F2(F3(y)));
```

where x, y, and z are shared integers and $\mathrm{F1}, \mathrm{F2}, \mathrm{F3}$ are functions.
Our job is to write the synchronization code to force the expression to be evaluated regardless of the order they are run by the CPU scheduler.

void T1(){
 	
y = F3(y);
 	
x=F1(x);
 	
z = F3(x,y);
 	
}
	void T2(){
	
	y = F2(y);
	
	}

What will be the minimum number of binary semaphores to make sure of this?

Your Answer: | Correct Answer: 2 | Not Attempted | Time taken: 00min 00sec | Discuss

Q #16 Multiple Choice Type | Award: 2 | Penalty: 0.67 | Operating System

Consider a system that has many threads of two difference types (typeA and typeB). All the threads call the below subroutine named *routine* passing their type as an argument.

```
Semaphore semaX = 0;   // Initial value of semaX is zero.
Semaphore semaY = 0;   // Initial value of semaY is zero.

void routine(threadType)
{
   if (threadType == typeA) {
      P(semaX);
      V(semaY);
   }
   if (threadType == typeB) {
      V(semaX);
      V(semaX);
      P(semaY);
   }
   DoIt();
}
```

Let 'a' represent the number of typeA threads that pass through the 'DoIt()' function, and 'b' represent the number of typeB threads that pass through the 'DoIt()' function. What relationship must ALWAYS hold true at any point in the system?

A. $a < 2b$
B. $b < 2a$
C. $b \leq a \leq 2b$
D. $a \leq b \leq 2a$

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Copyright & Stuff